

# AIPL<sub>max</sub> の文法・型健全性・型安全性

— デッドロック自由を目標から外したとき、言語はどこまで広げられるか —

児玉工房 / AICE・AIPL プロジェクト

2026 年 08 月 24 日

## 概要

第 3 版は**義務レベル**を導入して、`await` を取り除かずにデッドロック自由まで機械検証した。その代償として言語は狭い — 待ちは必ずレベルの高い方へ向かわねばならず、返答の委譲も、待ちの環も、自分自身への `now` も書けない。

本稿は逆向きに進む。目標を**型健全性と型安全性の二つだけ**に絞り、その二つが成り立つ範囲で言語を**最大まで広げた断片** AIPL<sub>max</sub> を定義する。AIPL<sup>-2</sup> から義務レベルと生産者不変条件を取り除き、代わりに実装 AIPL にある機構を五つ加えた — **文字列と ++、対、`result`( $\tau$ )、期限付きの待ち** (`timeout ... else` と `timeout`)、そして**一級の返答先** `reply`( $\tau$ ) (`replyto` と `answer` による返答の委譲) である。待ちには側条件を一切課さない — 自分自身を待ってもよいし、環を作ってもよい。

構文・型・意味論を与え、**紙の上で保存・進行・型安全性**を証明し、同じ内容を Rocq (Coq) 9.1.0 で機械検証した。本稿で新たに機械検証した定理は 15 本、いずれも `Print Assumptions` が `Closed under the global context` (公理なし) である。

「最大である」ことは三つの形で示す。

- 加えた五つの機構が、いずれも健全性を壊さないこと (第 II 部の証明)。
- さらに加えると健全性が**実際に破れる**機構の一覧と、破れ方 (第 IV 部)。動的な宛先の `remote`、`any`、計算されたメソッド名、範囲検査のない添字、注釈のない `reply`。
- この断片では**デッドロック自由が成り立たない**ことを、型の付いた構成でありながら全タスクが待ち合ったまま止まる具体例で示す (機械検証済み、`max_admits_deadlock`)。それでも型安全性は失われない — 待っているのであって、壊れてはいない。

## 目次

|       |                         |   |
|-------|-------------------------|---|
| 第 I 部 | AIPL <sub>max</sub> の定義 | 3 |
| 1     | 位置づけ                    | 3 |
| 1.1   | 三つの断片 . . . . .         | 3 |
| 1.2   | 何を証明するのか . . . . .      | 4 |
| 2     | 文法                      | 4 |
| 2.1   | 型 . . . . .             | 4 |
| 2.2   | 式 . . . . .             | 4 |
| 2.3   | 値 . . . . .             | 5 |
| 2.4   | 糖衣構文 . . . . .          | 5 |

|                        |                                      |    |
|------------------------|--------------------------------------|----|
| 2.5                    | プログラム . . . . .                      | 5  |
| 3                      | <b>型システム</b>                         | 5  |
| 3.1                    | 判断の形 . . . . .                       | 5  |
| 3.2                    | 規則 . . . . .                         | 6  |
| 3.3                    | プログラムの整合性 . . . . .                  | 7  |
| 3.4                    | 設計上の要点 . . . . .                     | 7  |
| 4                      | <b>操作的意味論</b>                        | 8  |
| 4.1                    | 実行時の構造 . . . . .                     | 8  |
| 4.2                    | 評価文脈 . . . . .                       | 8  |
| 4.3                    | 局所簡約 . . . . .                       | 8  |
| 4.4                    | 構成の簡約 . . . . .                      | 10 |
| 4.5                    | 待ち・終状態・行き詰まり . . . . .               | 11 |
| 5                      | <b>実行時の不変条件</b>                      | 11 |
| <b>第 II 部 紙の上の証明</b>   |                                      | 11 |
| 6                      | <b>準備の補題</b>                         | 12 |
| 7                      | <b>局所進行</b>                          | 13 |
| 8                      | <b>局所保存</b>                          | 14 |
| 9                      | <b>主定理</b>                           | 16 |
| 10                     | <b>派生する不変条件</b>                      | 17 |
| <b>第 III 部 機械証明</b>    |                                      | 18 |
| 11                     | <b>開発の構成</b>                         | 18 |
| 12                     | <b>紙と機械の対応</b>                       | 18 |
| 13                     | <b>仮定が空虚でないこと</b>                    | 20 |
| 14                     | <b>公理なしの確認と再現手順</b>                  | 20 |
| <b>第 IV 部 なぜこれが上限か</b> |                                      | 21 |
| 15                     | <b>加えた機構</b>                         | 21 |
| 16                     | <b>加えると壊れるもの</b>                     | 22 |
| 16.1                   | 動的な宛先の <code>remote</code> . . . . . | 22 |

|      |                            |    |
|------|----------------------------|----|
| 16.2 | any と動的キャスト . . . . .      | 23 |
| 16.3 | 計算されたメソッド名 . . . . .       | 23 |
| 16.4 | 範囲検査のない添字 . . . . .        | 23 |
| 16.5 | 注釈のない reply . . . . .      | 23 |
| 16.6 | replyto を値にすること . . . . .  | 23 |
| 16.7 | 受け取る型が動的に決まる選択受信 . . . . . | 23 |
| 17   | 直交する層                      | 23 |
| 18   | デッドロック自由は、ここでは成り立たない       | 24 |
| 19   | まとめ                        | 25 |

## 第 I 部

# AIPL<sub>max</sub><sup>-</sup> の定義

## 1 位置づけ

### 1.1 三つの断片

AIPL の形式化には、目標の違う三つの断片がある。表 1 にまとめる。

表 1 三つの断片。本稿は右端である。

|           | AIPL <sup>-</sup> (第 1 版) | AIPL <sup>-2</sup> (第 3 版) | AIPL <sub>max</sub> <sup>-</sup> (本稿) |
|-----------|---------------------------|----------------------------|---------------------------------------|
| 目標        | 健全性・安全性                   | +デッドロック自由                  | 健全性・安全性                               |
| await     | 言語から除外                    | 側条件つきで許す                   | 無条件に許す                                |
| 義務レベル     | なし                        | あり (待ちは上へ)                 | なし                                    |
| 生産者不変条件   | なし                        | あり                         | なし                                    |
| 効果        | なし                        | あり                         | なし (直交・第 3 版で証明済み)                    |
| 文字列・対     | なし                        | なし                         | あり                                    |
| result・期限 | なし                        | なし                         | あり                                    |
| 一級の返答先    | なし                        | なし                         | あり (委譲できる)                            |
| デッドロック自由  | —                         | 定理                         | 反例あり                                  |

第 3 版は「待ちを取り除かずにデッドロック自由を証明する」ために、future( $\tau$ ) の型にレベルを載せ、await に  $L < n$  という側条件を課した。これは強い制限である。返答の委譲 — 自分の返答先を他人に渡して、その他人に答えてもらう — は、返す義務がレベルの低い側へ移るので書けない。実装 AIPL はこれを許している (replyto / answer)。

本稿の問いは次のものである。

デッドロック自由を諦めるなら、型健全性と型安全性を保ったまま、言語はどこまで広げられるか。

## 1.2 何を証明するのか

用語を先に固定する。

**型健全性 (type soundness)** **保存** (型の付いた構成が一步進んでも型が付いたまま) と **進行** (型の付いた構成は、終わっているか、一步進めるか、待っているかのいずれか) の組をいう。

**型安全性 (type safety)** 型の付いた構成から到達できるどの構成も **行き詰まらない** こと。行き詰まる (stuck) とは、終状態でも待ち状態でもないのに一步も進めないことをいう。メソッド不理解、型の食い違い、存在しない actor や future への操作はすべてここに落ちる。

**デッドロック自由 (deadlock freedom)** 待ち状態 (blocked) が起こらないこと。本稿では証明しない。それどころか、成り立たないことを示す。

型安全性が「待ち状態を除いて行き詰まらない」という形になるのは、待ちを無条件に許した以上、避けられない。第 IV 部 §18 で、その線引きが恣意的でないこと (待ちは「壊れている」のではなく「まだ答えが来ていない」だけであること) を、反例の性質として確かめる。

## 2 文法

### 2.1 型

$$\begin{aligned} \tau ::= & \text{int} \mid \text{bool} \mid \text{unit} \mid \text{str} \mid \text{actor}[c] \mid (\tau_1 \times \tau_2) \\ & \mid \text{future}\langle\tau\rangle \mid \text{reply}\langle\tau\rangle \mid \text{result}\langle\tau\rangle \end{aligned}$$

$c$  はクラス番号である。三つの  $\langle\cdot\rangle$  型が、この断片の要点である。

$\text{future}\langle\tau\rangle$  「いつか  $\tau$  の値が入る箱」。  $\text{future } t.m(e)$  が作る。

$\text{reply}\langle\tau\rangle$  「 $\tau$  の値を入れる **権利**」。  $\text{replyto}$  が作り、  $\text{answer}$  が使う。  $\text{future}\langle\tau\rangle$  の裏側であり、**値として持ち運べる**。これが返答の委譲を可能にする。

$\text{result}\langle\tau\rangle$  「成功なら  $\tau$ 、失敗なら何もない」。期限付きの待ちが失敗しうることを型に出す。

### 2.2 式

$$\begin{aligned} e ::= & n \mid b \mid () \mid s \mid x \mid \text{self} \mid \text{replyto} \\ & \mid e_1 + e_2 \mid e_1 ++ e_2 \mid e_1 < e_2 \\ & \mid \text{if } e \text{ then } e_1 \text{ else } e_2 \mid \text{var } x = e_1; e_2 \mid e_1; e_2 \mid \text{while } e_1 \text{ do } e_2 \\ & \mid \text{state} \mid \text{state} := e \mid \text{new } c \\ & \mid \langle e_1, e_2 \rangle \mid e.\text{fst} \mid e.\text{snd} \\ & \mid \text{ok}(e) \mid \text{err}_\tau \mid e.\text{is\_ok} \mid e.\text{value}(e_d) \\ & \mid \text{future } e_a.m(e_1) \mid \text{now } e \mid \text{now } e \text{ timeout } n \text{ else } e_d \mid \text{now } e \text{ timeout } n \\ & \mid \text{answer } e_r \ e_v \\ & \mid \iota_o \mid \kappa_k \mid \rho_k \quad (\text{実行時のみ現れる}) \end{aligned}$$

$n$  は自然数、 $b$  は真偽値、 $s$  は文字列、 $x$  は変数、 $c$  はクラス番号、 $m$  はメソッド番号である。最後の行は**実行時項**で、原始プログラムには書けない。 $\iota_o$  は actor 参照、 $\kappa_k$  は future 参照、 $\rho_k$  は返答先参照である。 $\kappa_k$  と  $\rho_k$  は同じ番号  $k$  を指すが、型が違う —  $\kappa_k : \text{future}\langle\tau\rangle$  (読む側)、 $\rho_k : \text{reply}\langle\tau\rangle$  (書く側)。

## 2.3 値

$$v ::= n \mid b \mid () \mid s \mid \iota_o \mid \kappa_k \mid \rho_k \mid \langle v_1, v_2 \rangle \mid \text{ok}(v) \mid \text{err}_\tau$$

## 2.4 糖衣構文

実装 AIPL の表記との対応は表 2 のとおりである。糖衣はすべて上の式に展開できるので、規則を増やす必要はない。

表 2 AIPL の表記と  $\text{AIPL}_{\max}^-$  の式の対応

| AIPL の表記                                   | $\text{AIPL}_{\max}^-$ での意味                                                             |
|--------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>send t.m(e)</code>                   | <code>var x = future t.m(e); ()</code> ( $x$ は使わない)                                     |
| <code>now t.m(e)</code>                    | <code>now (future t.m(e))</code>                                                        |
| <code>now t.m(e) timeout 500 else d</code> | <code>now (future t.m(e)) timeout 500 else d</code>                                     |
| <code>now t.m(e) timeout 500</code>        | <code>now (future t.m(e)) timeout 500</code> ( <code>result(<math>\tau</math>)</code> ) |
| <code>reply e</code>                       | <code>answer replyto e</code>                                                           |
| <code>m(a, b)</code>                       | 対を一つ渡す <code>m(<math>\langle a, b \rangle</math>)</code>                                |
| <code>r.value</code>                       | <code>r.value(d)</code> (既定値 $d$ つき。§16 参照)                                             |
| 複数のフィールド                                   | 対の入れ子                                                                                   |

## 2.5 プログラム

プログラムは四つの表で与える。クラスもメソッドも番号で表す。

|                                                     |                  |
|-----------------------------------------------------|------------------|
| $\text{st} : c \mapsto \tau$                        | クラス $c$ の状態の型    |
| $\text{si} : c \mapsto v$                           | クラス $c$ の状態の初期値  |
| $\text{mt} : (c, m) \mapsto (\tau_a, \tau_r) \perp$ | メソッドの引数型と返り値型    |
| $\text{mb} : (c, m) \mapsto e$                      | メソッド本体 (引数は変数 0) |

加えて、起動時に居る actor の表  $\Omega_0$  を固定する。 $\text{mt}(c, m) = \perp$  は「クラス  $c$  にメソッド  $m$  が無い」ことを表す。

# 3 型システム

## 3.1 判断の形

判断は

$$\Omega; \Phi; c; \rho; \Gamma \vdash e : \tau$$

の形をとる。読み方は次のとおりである。

- $\Omega$  actor 表。  $\Omega(o)$  は actor  $o$  のクラス。
- $\Phi$  future 表。  $\Phi(k)$  は future  $k$  に入る値の型。
- $c$  いま自分がいるクラス。 **self** と **state** の型を決める。
- $\rho$  **いま実行中のメソッドの返り値型**。 **replyto** の型を決める。
- $\Gamma$  変数の型環境。

$\rho$  を判断に持つのが  $\text{AIPL}^{-2}$  との違いである。  $\text{AIPL}^{-2}$  は同じ位置に義務レベル  $L$  を持っていた。**レベルを外して、返り値型を入れた** — これが本稿の一行の要約である。

以下、 $\Omega; \Phi; c; \rho$  は規則を通して変わらないので省略し、 $\Gamma \vdash e : \tau$  と書く。省略した成分を使う規則だけは、 $\Omega(o) = c'$  のように明示する。

### 3.2 規則

$$\begin{array}{c}
\frac{}{\Gamma \vdash n : \text{int}} \text{T-Num} \quad \frac{}{\Gamma \vdash b : \text{bool}} \text{T-Bool} \quad \frac{}{\Gamma \vdash () : \text{unit}} \text{T-Unit} \quad \frac{}{\Gamma \vdash s : \text{str}} \text{T-Str} \\
\\
\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \text{T-Var} \quad \frac{}{\Gamma \vdash \text{self} : \text{actor}[c]} \text{T-Self} \quad \frac{\Omega(o) = c'}{\Gamma \vdash \iota_o : \text{actor}[c']} \text{T-OREF} \\
\\
\frac{\Phi(k) = \tau}{\Gamma \vdash \kappa_k : \text{future}\langle\tau\rangle} \text{T-FREF} \quad \frac{\Phi(k) = \tau}{\Gamma \vdash \rho_k : \text{reply}\langle\tau\rangle} \text{T-RREF}
\end{array}$$

#### ■演算と制御

$$\begin{array}{c}
\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \text{T-Add} \quad \frac{\Gamma \vdash e_1 : \text{str} \quad \Gamma \vdash e_2 : \text{str}}{\Gamma \vdash e_1 ++ e_2 : \text{str}} \text{T-Cat} \\
\\
\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 < e_2 : \text{bool}} \text{T-Lt} \quad \frac{\Gamma \vdash e : \text{bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \text{if } e \text{ then } e_1 \text{ else } e_2 : \tau} \text{T-If} \\
\\
\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x:\tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{var } x = e_1; e_2 : \tau_2} \text{T-Let} \quad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1; e_2 : \tau_2} \text{T-Seq} \\
\\
\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash \text{while } e_1 \text{ do } e_2 : \text{unit}} \text{T-While}
\end{array}$$

## ■状態・生成・対・result

$$\begin{array}{c}
\frac{}{\Gamma \vdash \text{state} : \text{st}(c)} \text{T-GET} \quad \frac{\Gamma \vdash e : \text{st}(c)}{\Gamma \vdash \text{state} := e : \text{unit}} \text{T-SET} \quad \frac{}{\Gamma \vdash \text{new } c' : \text{actor}[c']} \text{T-NEW} \\
\\
\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash \langle e_1, e_2 \rangle : (\tau_1 \times \tau_2)} \text{T-PAIR} \quad \frac{\Gamma \vdash e : (\tau_1 \times \tau_2)}{\Gamma \vdash e.\text{fst} : \tau_1} \text{T-FST} \quad \frac{\Gamma \vdash e : (\tau_1 \times \tau_2)}{\Gamma \vdash e.\text{snd} : \tau_2} \text{T-SND} \\
\\
\frac{\Gamma \vdash e : \tau}{\Gamma \vdash \text{ok}(e) : \text{result}\langle\tau\rangle} \text{T-OK} \quad \frac{}{\Gamma \vdash \text{err}_\tau : \text{result}\langle\tau\rangle} \text{T-ERR} \quad \frac{\Gamma \vdash e : \text{result}\langle\tau\rangle}{\Gamma \vdash e.\text{is\_ok} : \text{bool}} \text{T-ISOK} \\
\\
\frac{\Gamma \vdash e : \text{result}\langle\tau\rangle \quad \Gamma \vdash e_d : \tau}{\Gamma \vdash e.\text{value}(e_d) : \tau} \text{T-VALUE}
\end{array}$$

## ■通信と返答

$$\begin{array}{c}
\frac{\Gamma \vdash e_a : \text{actor}[c'] \quad \text{mt}(c', m) = (\tau_a, \tau_r) \quad \Gamma \vdash e_1 : \tau_a}{\Gamma \vdash \text{future } e_a.m(e_1) : \text{future}\langle\tau_r\rangle} \text{T-SEND} \quad \frac{\Gamma \vdash e : \text{future}\langle\tau\rangle}{\Gamma \vdash \text{now } e : \tau} \text{T-AWAIT} \\
\\
\frac{\Gamma \vdash e : \text{future}\langle\tau\rangle \quad \Gamma \vdash e_d : \tau}{\Gamma \vdash \text{now } e \text{ timeout } n \text{ else } e_d : \tau} \text{T-AWAITD} \quad \frac{\Gamma \vdash e : \text{future}\langle\tau\rangle}{\Gamma \vdash \text{now } e \text{ timeout } n : \text{result}\langle\tau\rangle} \text{T-AWAITR} \\
\\
\frac{}{\Gamma \vdash \text{replyto} : \text{reply}\langle\rho\rangle} \text{T-REPLYTO} \quad \frac{\Gamma \vdash e_r : \text{reply}\langle\tau\rangle \quad \Gamma \vdash e_v : \tau}{\Gamma \vdash \text{answer } e_r \text{ } e_v : \text{unit}} \text{T-ANSWER}
\end{array}$$

## 3.3 プログラムの整合性

**定義 1** (整合したプログラム). プログラム  $(\text{st}, \text{si}, \text{mt}, \text{mb}, \Omega_0)$  が**整合している**とは、次の三つが成り立つことをいう。

- (P1) すべての  $c$  について  $\text{si}(c)$  は値である。
- (P2) すべての  $c$  について、任意の  $\Omega, \Phi, c', \rho, \Gamma$  のもとで  $\Omega; \Phi; c'; \rho; \Gamma \vdash \text{si}(c) : \text{st}(c)$ 。
- (P3)  $\text{mt}(c, m) = (\tau_a, \tau_r)$  なるすべての  $c, m$  と、 $\Omega_0 \sqsubseteq \Omega$  なるすべての  $\Omega$ 、すべての  $\Phi$  について

$$\Omega; \Phi; c; \tau_r; \{0; \tau_a\} \vdash \text{mb}(c, m) : \tau_r.$$

ここで  $\Omega_0 \sqsubseteq \Omega$  は「 $\Omega$  が  $\Omega_0$  の後ろへの延長である」ことを表す。

(P3) で  $\rho = \tau_r$  と置いているのが要点である。**メソッド本体の中では、replyto の型はそのメソッドの返り値型である。**実装 AIPL の型検査器も同じことをしている。以降のすべての定理は、この三条件を仮定して述べる。公理ではない — 第 III 部 §13 で、三条件を満たす具体的なプログラムを与えて、仮定が空虚でないことを確かめる。

## 3.4 設計上の要点

**待ちに側条件がない** T-AWAIT には前提が一つしかない。自分自身の future を待ってもよいし、二人が互いを待ってもよい。AIPL<sup>-2</sup> の  $L < n$  に当たるものが無い。これが AIPL<sub>max</sub><sup>-</sup> を広くしている — そして、デッドロック自由を失わせている。

**+ と ++ を型で分ける** T-ADD は `int` だけ、T-CAT は `str` だけを受ける。実装 AIPL がこの区別を導入した理由（数と文字列の暗黙変換をやめる）が、そのまま型に出ている。

**返答先が値である** T-RREF により  $\rho_k$  は値であり、T-SEND の引数として渡せる。渡された先は T-ANSWER で答えられる。**答える者と、呼ばれた者は、別人でよい。**

**期限は型に出る** 既定値付きの待ち（T-AWAITD）は  $\tau$  を返し、既定値なしの待ち（T-AWAITR）は `result $\langle\tau\rangle$`  を返す。失敗しうることが型に現れるのは後者だけである。

## 4 操作的意味論

### 4.1 実行時の構造

**定義 2** (ヒープ・メッセージ・タスク・構成).

|                                    |                                                                                                                 |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| $H = (\Omega, \Sigma, \Phi, \Psi)$ | $\Omega : o \mapsto c, \quad \Sigma : o \mapsto v, \quad \Phi : k \mapsto \tau, \quad \Psi : k \mapsto v_\perp$ |
| $M = \langle o, m, v, k \rangle$   | 宛先 $o$ 、メソッド $m$ 、引数 $v$ 、返答先の future $k$                                                                       |
| $t = \langle o, k, e \rangle$      | self が $o$ 、埋めるべき future が $k$ 、実行中の式が $e$                                                                      |
| $C = (H, \vec{M}, \vec{t})$        | 構成                                                                                                              |

$\Psi(k) = \perp$  は「future  $k$  はまだ解決していない」ことを表す。四つの表はいずれも後ろへ伸びるだけで、 $\Omega$  と  $\Phi$  の既存の項目は書き換わらない。 $\Sigma$  は `state :=` で、 $\Psi$  は解決で書き換わる。

### 4.2 評価文脈

$$\begin{aligned}
E ::= & [] \mid E + e \mid v + E \mid E ++ e \mid v ++ E \mid E < e \mid v < E \\
& \mid \text{if } E \text{ then } e_1 \text{ else } e_2 \mid \text{var } x = E; e \mid E; e \mid \text{state} := E \\
& \mid \langle E, e \rangle \mid \langle v, E \rangle \mid E.\text{fst} \mid E.\text{snd} \\
& \mid \text{ok}(E) \mid E.\text{is\_ok} \mid E.\text{value}(e) \\
& \mid \text{future } E.m(e) \mid \text{future } v.m(E) \\
& \mid \text{now } E \mid \text{now } E \text{ timeout } n \text{ else } e \mid \text{now } E \text{ timeout } n \\
& \mid \text{answer } E \ e \mid \text{answer } v \ E
\end{aligned}$$

評価は左から右、値になってから次へ進む。if の枝、var の本体、while の本体、value の既定値、期限付き待ちの既定値は**文脈に入らない**— 必要になるまで評価しない。

### 4.3 局所簡約

判断は  $H; o; k \vdash e \longrightarrow H'; \vec{M}; e'$  である。 $o$  は実行中のタスクの self、 $k$  はそのタスクが埋める future、 $\vec{M}$  はこの一步で送り出されたメッセージである。



$$\begin{array}{c}
\frac{}{H; o; k \vdash n_1 + n_2 \longrightarrow H; \emptyset; n_1 + n_2} \text{E-ADD} \qquad \frac{}{H; o; k \vdash s_1 ++ s_2 \longrightarrow H; \emptyset; s_1 s_2} \text{E-CAT} \\
\\
\frac{}{H; o; k \vdash n_1 < n_2 \longrightarrow H; \emptyset; (n_1 <_{\mathbb{N}} n_2)} \text{E-LT} \\
\\
\frac{}{H; o; k \vdash \text{if true then } e_1 \text{ else } e_2 \longrightarrow H; \emptyset; e_1} \text{E-IFT} \\
\\
\frac{}{H; o; k \vdash \text{if false then } e_1 \text{ else } e_2 \longrightarrow H; \emptyset; e_2} \text{E-IFF} \\
\\
\frac{}{H; o; k \vdash \text{var } x = v; e \longrightarrow H; \emptyset; e[x \mapsto v]} \text{E-LET} \qquad \frac{}{H; o; k \vdash v; e \longrightarrow H; \emptyset; e} \text{E-SEQ} \\
\\
\frac{}{H; o; k \vdash \text{while } e_1 \text{ do } e_2 \longrightarrow H; \emptyset; \text{if } e_1 \text{ then } (e_2; \text{while } e_1 \text{ do } e_2) \text{ else } ()} \text{E-WHILE}
\end{array}$$

### ■ 自分・状態・生成

$$\begin{array}{c}
\frac{}{H; o; k \vdash \text{self} \longrightarrow H; \emptyset; \iota_o} \text{E-SELF} \qquad \frac{}{H; o; k \vdash \text{replyto} \longrightarrow H; \emptyset; \rho_k} \text{E-REPLYTO} \\
\\
\frac{\Sigma(o) = v}{H; o; k \vdash \text{state} \longrightarrow H; \emptyset; v} \text{E-GET} \qquad \frac{}{H; o; k \vdash \text{state} := v \longrightarrow (\Omega, \Sigma[o \mapsto v], \Phi, \Psi); \emptyset; ()} \text{E-SET} \\
\\
\frac{}{H; o; k \vdash \text{new } c' \longrightarrow (\Omega \cdot c', \Sigma \cdot \text{si}(c'), \Phi, \Psi); \emptyset; \iota_{|\Omega|}} \text{E-NEW}
\end{array}$$

### ■ 対と result

$$\begin{array}{c}
\frac{}{H; o; k \vdash \langle v_1, v_2 \rangle.\text{fst} \longrightarrow H; \emptyset; v_1} \text{E-FST} \qquad \frac{}{H; o; k \vdash \langle v_1, v_2 \rangle.\text{snd} \longrightarrow H; \emptyset; v_2} \text{E-SND} \\
\\
\frac{}{H; o; k \vdash \text{ok}(v).\text{is\_ok} \longrightarrow H; \emptyset; \text{true}} \text{E-ISOKT} \\
\\
\frac{}{H; o; k \vdash \text{err}_\tau.\text{is\_ok} \longrightarrow H; \emptyset; \text{false}} \text{E-ISOKF} \\
\\
\frac{}{H; o; k \vdash \text{ok}(v).\text{value}(e_d) \longrightarrow H; \emptyset; v} \text{E-VALOK} \\
\\
\frac{}{H; o; k \vdash \text{err}_\tau.\text{value}(e_d) \longrightarrow H; \emptyset; e_d} \text{E-VALERR}
\end{array}$$

## ■通信と返答

$$\begin{array}{c}
\frac{\Omega(o') = c' \quad \text{mt}(c', m) = (\tau_a, \tau_r)}{H; o; k \vdash \text{future } \iota_{o'}.m(v) \longrightarrow (\Omega, \Sigma, \Phi \cdot \tau_r, \Psi \cdot \perp); \{\langle o', m, v, |\Phi| \rangle\}; \kappa_{|\Phi|}} \text{E-SEND} \\
\\
\frac{\Psi(k') = v}{H; o; k \vdash \text{now } \kappa_{k'} \longrightarrow H; \emptyset; v} \text{E-AWAIT} \\
\\
\frac{\Psi(k') = v}{H; o; k \vdash \text{now } \kappa_{k'} \text{ timeout } n \text{ else } e_d \longrightarrow H; \emptyset; v} \text{E-AWAITDV} \\
\\
\frac{\Phi(k') = \tau}{H; o; k \vdash \text{now } \kappa_{k'} \text{ timeout } n \text{ else } e_d \longrightarrow H; \emptyset; e_d} \text{E-AWAITDT} \\
\\
\frac{\Psi(k') = v}{H; o; k \vdash \text{now } \kappa_{k'} \text{ timeout } n \longrightarrow H; \emptyset; \text{ok}(v)} \text{E-AWAITRV} \\
\\
\frac{\Phi(k') = \tau}{H; o; k \vdash \text{now } \kappa_{k'} \text{ timeout } n \longrightarrow H; \emptyset; \text{err}_\tau} \text{E-AWAITRT} \\
\\
\frac{}{H; o; k \vdash \text{answer } \rho_{k'} v \longrightarrow (\Omega, \Sigma, \Phi, \Psi[k' \mapsto v]); \emptyset; ()} \text{E-ANSWER} \\
\\
\frac{H; o; k \vdash e \longrightarrow H'; \vec{M}; e'}{H; o; k \vdash E[e] \longrightarrow H'; \vec{M}; E[e']} \text{E-CTX}
\end{array}$$

注意 3 (期限の扱い). E-AWAITDT と E-AWAITRT はいいつでも適用できる。\$n\$ は数として保持するだけで、簡約の条件にはしない。時計を持たない意味論で「期限が切れうる」ことを表す、いちばん弱い (=安全側の) 定式化である。実装は時計を見て切るので、実装の振る舞いはこの意味論の振る舞いの一部になっている。

注意 4 (答えの上書き). E-ANSWER は、すでに解決している future にも書ける (上書きする)。実装は二度目の answer を実行時に拒むが、**拒む方が振る舞いが少ない**。上書きを許した方の意味論で定理を証明しておけば、実装の振る舞いは覆われる。上書きされても型は変わらない (定理 28)。

## 4.4 構成の簡約

$$\begin{array}{c}
\frac{H; o; k \vdash e \longrightarrow H'; \vec{M}'; e'}{(H, \vec{M}, \vec{t}_1 \langle o, k, e \rangle \vec{t}_2) \Longrightarrow (H', \vec{M} \vec{M}', \vec{t}_1 \langle o, k, e' \rangle \vec{t}_2)} \text{C-TASK} \\
\\
\frac{}{(H, \vec{M}, \vec{t}_1 \langle o, k, v \rangle \vec{t}_2) \Longrightarrow ((\Omega, \Sigma, \Phi, \Psi[k \mapsto v]), \vec{M}, \vec{t}_1 \vec{t}_2)} \text{C-FINISH} \\
\\
\frac{\Omega(o) = c \quad \text{mt}(c, m) = (\tau_a, \tau_r)}{(H, \vec{M}_1 \langle o, m, v, k \rangle \vec{M}_2, \vec{t}) \Longrightarrow (H, \vec{M}_1 \vec{M}_2, \vec{t} \cdot \langle o, k, \text{mb}(c, m)[0 \mapsto v] \rangle)} \text{C-DELIVER}
\end{array}$$

\$\Longrightarrow^\*\$ を \$\Longrightarrow\$ の反射推移閉包とする。

## 4.5 待ち・終状態・行き詰まり

定義 5.

$$\begin{aligned} \text{awaiting}(H, e) &\iff \exists E, k'. e = E[\text{now } \kappa_{k'}] \wedge \Psi(k') = \perp \\ \text{terminal}(C) &\iff \vec{M} = \emptyset \wedge \vec{t} = \emptyset \\ \text{blocked}(C) &\iff \vec{M} = \emptyset \wedge \vec{t} \neq \emptyset \wedge \forall \langle o, k, e \rangle \in \vec{t}. \text{awaiting}(H, e) \\ \text{stuck}(C) &\iff \neg \text{terminal}(C) \wedge \neg \text{blocked}(C) \wedge \neg \exists C'. C \implies C' \end{aligned}$$

`awaiting` に入るのは期限のない待ちだけである。`now  $\kappa_{k'}$  timeout  $n$  else  $e_d$`  は、 $k'$  が未解決でも E-AWAITDT で一歩進めるので、待ち状態ではない（定理 29）。

## 5 実行時の不変条件

定義 6 (`heap_ok`). `heap_ok(H)` とは、次の四つが成り立つことをいう。

1.  $|\Sigma| = |\Omega|$  かつ  $|\Psi| = |\Phi|$ 。
2.  $\Omega(o) = c$  かつ  $\Sigma(o) = v$  ならば、 $v$  は値であり、任意の  $c', \rho, \Gamma$  のもとで  $\Omega; \Phi; c'; \rho; \Gamma \vdash v : \text{st}(c)$ 。
3.  $\Phi(k) = \tau$  かつ  $\Psi(k) = v$  (解決済み) ならば、 $v$  は値であり、任意の  $c', \rho, \Gamma$  のもとで  $\Omega; \Phi; c'; \rho; \Gamma \vdash v : \tau$ 。

(2 と 3 で「任意の文脈で」と言えるのは、値の型付けが文脈に依らないからである — 補題 13。)

定義 7 (`msg_ok`). `msg_ok(H, \langle o, m, v, k \rangle)` とは、 $\Omega(o) = c$ 、 $\text{mt}(c, m) = (\tau_a, \tau_r)$ 、 $v$  は値、任意の文脈で  $\vdash v : \tau_a$ 、そして  $\Phi(k) = \tau_r$  が成り立つことをいう。

飛んでいるメッセージについて、宛先にそのメソッドがあることと、返答先 `future` の型が返り値型と一致していることを要求している。前者が「メソッド不理解が起きない」（定理 21）の中身である。

定義 8 (`task_ok`). `task_ok(H, \langle o, k, e \rangle)` とは、 $\Omega(o) = c$ 、 $\Phi(k) = \tau$  であって

$$\Omega; \Phi; c; \tau; \emptyset \vdash e : \tau$$

が成り立つことをいう。

判断の第 4 成分 ( $\rho$ ) と結論の型が、どちらも  $\Phi(k)$  になっている。**タスクが返す型と、`replyto` の型は、同じ `future` の型である。**これが委譲を型安全にしている要である。

定義 9 (`conf_ok`). `conf_ok(H, \vec{M}, \vec{t})` とは、`heap_ok(H)`、 $\Omega_0 \sqsubseteq \Omega$ 、 $\vec{M}$  のすべての要素が `msg_ok`、 $\vec{t}$  のすべての要素が `task_ok` であることをいう。

注意 10 (何を要求していないか). AIPL<sup>-2</sup> にあった**生産者不変条件**「未解決の `future` には必ずそれを埋める者（メッセージかタスク）がいる」を、AIPL<sub>max</sub><sup>-</sup> は要求しない。孤児 `future` — 誰も埋めない `future` — を許す。許してよいのは、健全性も安全性も孤児 `future` で壊れないからである（壊れるのはデッドロック自由だけである）。実際、§18 の反例は生産者を持つが、待ちの向きが環になっているために止まる。

## 第 II 部

# 紙の上の証明

以下、プログラムは定義 1 の意味で整合しているものとする。 $\Omega \sqsubseteq \Omega'$  は「 $\Omega'$  が  $\Omega$  の後ろへの延長」を表す。

## 6 準備の補題

**補題 11** (逆転). 型付け規則は式の頭の構成子ごとにちょうど一つなので、判断から導出の最後の規則が一意に決まる。たとえば

- $\Gamma \vdash e_1 + e_2 : \tau$  ならば  $\tau = \text{int}$  かつ  $\Gamma \vdash e_1 : \text{int}$  かつ  $\Gamma \vdash e_2 : \text{int}$ 。
- $\Gamma \vdash \text{future } e_a.m(e_1) : \tau$  ならば、ある  $c', \tau_a, \tau_r$  があって  $\Gamma \vdash e_a : \text{actor}[c']$ 、 $\text{mt}(c', m) = (\tau_a, \tau_r)$ 、 $\Gamma \vdash e_1 : \tau_a$ 、 $\tau = \text{future}\langle\tau_r\rangle$ 。
- $\Gamma \vdash \text{now } e : \tau$  ならば  $\Gamma \vdash e : \text{future}\langle\tau\rangle$ 。
- $\Gamma \vdash \text{answer } e_r.e_v : \tau$  ならば  $\tau = \text{unit}$  で、ある  $\tau_0$  があって  $\Gamma \vdash e_r : \text{reply}\langle\tau_0\rangle$ 、 $\Gamma \vdash e_v : \tau_0$ 。
- $\Gamma \vdash \text{replyto} : \tau$  ならば  $\tau = \text{reply}\langle\rho\rangle$ 。

残りの構成子についても同様である。

*Proof.* 規則の結論に現れる式の頭の構成子はすべて相異なる。よって導出の最後の規則が決まり、その前提がそのまま主張になる。  $\square$

**補題 12** (標準形).  $v$  が値で  $\Gamma \vdash v : \tau$  のとき、

| $\tau$                               | $v$ の形                                             |
|--------------------------------------|----------------------------------------------------|
| <b>int</b>                           | $n$                                                |
| <b>bool</b>                          | $b$                                                |
| <b>str</b>                           | $s$                                                |
| <b>actor</b> $[c']$                  | $\iota_o$ であって $\Omega(o) = c'$                    |
| $(\tau_1 \times \tau_2)$             | $\langle v_1, v_2 \rangle$ であって $v_1, v_2$ は値      |
| <b>future</b> $\langle\tau_0\rangle$ | $\kappa_k$ であって $\Phi(k) = \tau_0$                 |
| <b>reply</b> $\langle\tau_0\rangle$  | $\rho_k$ であって $\Phi(k) = \tau_0$                   |
| <b>result</b> $\langle\tau_0\rangle$ | $\text{ok}(w)$ ( $w$ は値) または $\text{err}_{\tau_0}$ |

*Proof.*  $v$  が値であることの導出について場合分けし、それぞれについて補題 11 を使う。たとえば  $\tau = \text{future}\langle\tau_0\rangle$  のとき、値のうち  $\text{future}\langle\cdot\rangle$  型を持ちうるのは  $\kappa_k$  だけである ( $\iota_o$  は  $\text{actor}[\cdot]$ 、 $\rho_k$  は  $\text{reply}\langle\cdot\rangle$ 、 $\text{ok}(w)$  と  $\text{err}$  は  $\text{result}\langle\cdot\rangle$ 、残りは基本型)。  $\square$

**補題 13** (値の型付けは文脈に依らない).  $v$  が値で  $\Omega; \Phi; c; \rho; \Gamma \vdash v : \tau$  ならば、任意の  $c', \rho', \Gamma'$  について  $\Omega; \Phi; c'; \rho'; \Gamma' \vdash v : \tau$ 。

*Proof.*  $v$  が値であることの導出に関する帰納法。値の型付けに使える規則は T-NUM, T-BOOL, T-UNIT, T-STR, T-OREF, T-FREF, T-RREF, T-PAIR, T-OK, T-ERR の十個しかない。このうち  $c$

を見るのは無く (T-SELF と T-GET は値ではない)、 $\rho$  を見るのも無く (T-REPLYTO は値 `replyto` を作らない — `replyto` は値ではなく、 $\rho_k$  へ簡約される式である)、 $\Gamma$  を見るのも無い (T-VAR は値ではない)。 $\langle v_1, v_2 \rangle$  と  $\text{ok}(w)$  は帰納法の仮定から従う。  $\square$

注意 14. `replyto` が値でないことが効いている。もし `replyto` 自身を値にすると、それを他の actor へ送れてしまい、送り先では  $\rho$  が違うので型が変わる — 補題 13 が壊れ、その先の代入補題も保存定理も壊れる。`replyto` は自分のところで  $\rho_k$  に簡約され、 $\rho_k$  の型は  $\Phi$  が決めるので、どこへ運ばれても変わらない。一級の返答先を安全にしているのは、この一段である。

**補題 15** (代入).  $\Gamma, x:\tau_1 \vdash e:\tau$  で、 $v$  が値、かつ任意の文脈で  $\vdash v:\tau_1$  ならば、 $\Gamma \vdash e[x \mapsto v]:\tau$ 。

*Proof.*  $e$  の構造に関する帰納法。 $e = x$  のときは仮定から  $v$  の型が  $\tau_1 = \tau$  であり、補題 13 により  $\Gamma$  のもとでも型が付く。 $e = y \neq x$  のときは代入が起こらない。 $e = (\text{var } y = e_1; e_2)$  のときは、 $y = x$  なら  $e_2$  に代入しない (内側の束縛が優先する) ので  $\Gamma, x:\tau_1, x:\tau'_1 = \Gamma, x:\tau'_1$  の外延性を使い、 $y \neq x$  なら型環境の二つの拡張が可換であることを使う。他の構成子は、部分式へ帰納法の仮定を適用して同じ規則で組み直すだけである。  $\square$

**補題 16** (表の拡張に対する単調性).  $\Omega; \Phi; c; \rho; \Gamma \vdash e:\tau$  で  $\Omega \sqsubseteq \Omega', \Phi \sqsubseteq \Phi'$  ならば  $\Omega'; \Phi'; c; \rho; \Gamma \vdash e:\tau$ 。

*Proof.* 型付けの導出に関する帰納法。表を見る規則は T-OREF, T-FREF, T-RREF の三つだけで、いずれも  $\Omega(o) = c'$  や  $\Phi(k) = \tau_0$  という索引である。後ろへの延長は既存の索引を変えないので、そのまま成り立つ。他の規則は帰納法の仮定を組み直すだけである。  $\square$

**補題 17** (文脈の分解と置換).  $\Omega; \Phi; c; \rho; \Gamma \vdash E[e]:\tau$  ならば、ある  $\tau'$  があって  $\Omega; \Phi; c; \rho; \Gamma \vdash e:\tau'$  であり、かつ  $\Omega \sqsubseteq \Omega', \Phi \sqsubseteq \Phi'$  なる任意の表と、 $\Omega'; \Phi'; c; \rho; \Gamma \vdash e':\tau'$  なる任意の  $e'$  について  $\Omega'; \Phi'; c; \rho; \Gamma \vdash E[e']:\tau$ 。

*Proof.*  $E$  の構造に関する帰納法。 $E = []$  は自明。それ以外は、いちばん外側の規則を補題 11 で剥がし、穴のある部分式へ帰納法の仮定を、穴のない部分式へ補題 16 を適用して、同じ規則で組み直す。  $\square$

## 7 局所進行

**補題 18** (局所進行).  $\text{heap\_ok}(H)$ 、 $\Omega(o) = c$ 、 $\Omega; \Phi; c; \rho; \emptyset \vdash e:\tau$  ならば、次のいずれかが成り立つ。

$$(i) e \text{ は値} \quad \vee \quad (ii) \exists H', \vec{M}, e'. H; o; k \vdash e \longrightarrow H'; \vec{M}; e' \quad \vee \quad (iii) \text{awaiting}(H, e)$$

*Proof.* 型付けの導出に関する帰納法。型環境が空であることを使う。

T-VAR 起こらない。 $\emptyset(x) = \perp$  である。

T-NUM など  $e$  は値。

T-SELF, T-REPLYTO, T-NEW, T-WHILE 無条件に一步進む (E-SELF, E-REPLYTO, E-NEW, E-WHILE)。

T-GET  $\Omega(o) = c$  と  $\text{heap\_ok}$  の (1) から  $o < |\Sigma|$  なので  $\Sigma(o)$  が存在し、E-GET で一步進む。

T-ADD 帰納法の仮定を  $e_1$  に適用する。値でないなら (ii) か (iii) がそのまま文脈へ持ち上がる ( $E = [] + e_2$ )。値なら標準形補題 12 から  $e_1 = n_1$ 。次に  $e_2$  に適用する。値なら  $e_2 = n_2$  で

E-ADD、そうでなければ文脈  $n_1 + []$  へ持ち上がる。T-CAT, T-LT, T-PAIR, T-ANSWER も同型である（ただし T-PAIR は両方が値のとき  $\langle v_1, v_2 \rangle$  自身が値になる）。

T-IF, T-LET, T-SEQ, T-SET, T-FST, T-SND, T-ISOK, T-VALUE, T-OK 部分式に帰納法の仮定を適用し、値になっていれば標準形補題で形を決めて基底規則を当てる。たとえば T-VALUE では  $e : \text{result}\langle \tau \rangle$  が値なら  $\text{ok}(w)$  か  $\text{err}_\tau$  のいずれかであり、E-VALOK か E-VALERR が当たる。**err でも行き詰まらないのは、既定値  $e_d$  を必ず伴わせたからである。**

T-SEND  $e_a$  が値なら標準形補題から  $e_a = \iota_{o'}$  で  $\Omega(o') = c'$ 。  $e_1$  が値なら、前提の  $\text{mt}(c', m) = (\tau_a, \tau_r)$  がそのまま E-SEND の前提を満たす。**メソッド不理解が起こらないのはここである** — 宛先のクラスは型から分かっており、そのクラスにメソッドがあることは型付けのときに確かめてある。

T-AWAIT  $e$  が値なら  $e = \kappa_{k'}$  で  $\Phi(k') = \tau$ 。  $\text{heap\_ok}$  の (1) から  $k' < |\Psi|$  なので  $\Psi(k')$  が定まる。解決済みなら E-AWAIT で一歩進み、未解決なら  $\text{awaiting}(H, \text{now } \kappa_{k'})$ （文脈  $[]$ ）である。**ここだけが (iii) を生む。**

T-AWAITD, T-AWAITR  $e$  が値なら  $e = \kappa_{k'}$  で  $\Phi(k')$  が存在するから、解決の有無によらず E-AWAITDT / E-AWAITRT が当たる。すなわち**必ず一歩進める**。 $e$  が値でないときは、部分式についての (ii) か (iii) が文脈へ持ち上がる。

□

## 8 局所保存

**補題 19** (局所保存).  $\text{heap\_ok}(H)$ 、 $\Omega(o) = c$ 、 $\Phi(k) = \rho$ 、 $\Omega; \Phi; c; \rho; \emptyset \vdash e : \tau$ 、 $H; o; k \vdash e \longrightarrow H'; \vec{M}'; e'$  ならば

1.  $\text{heap\_ok}(H')$ 、
2.  $\Omega \sqsubseteq \Omega'$  かつ  $\Phi \sqsubseteq \Phi'$ 、
3.  $\Omega'; \Phi'; c; \rho; \emptyset \vdash e' : \tau$ 、
4.  $\vec{M}'$  のすべての要素が  $\text{msg\_ok}(H')$ 。

*Proof.* 簡約の導出に関する帰納法。以下、 $\tau$  の情報は毎回、補題 11 で取り出す。

■**ヒープを変えない基底規則** E-ADD, E-CAT, E-LT, E-IFT/F, E-SEQ, E-WHILE, E-FST, E-SND, E-ISOKT/F, E-VALOK/ERR, E-AWAITDT。 (1)(2)(4) は自明 ( $H' = H$ 、 $\vec{M}' = \emptyset$ )。 (3) だけを見る。たとえば E-WHILE では、逆転補題から  $\tau = \text{unit}$ 、 $\vdash e_1 : \text{bool}$ 、 $\vdash e_2 : \tau_1$  が得られ、これらから T-IF, T-SEQ, T-WHILE を組んで  $\vdash \text{if } e_1 \text{ then } (e_2; \text{while } e_1 \text{ do } e_2) \text{ else } () : \text{unit}$  を得る。E-VALOK では  $\vdash \text{ok}(v) : \text{result}\langle \tau \rangle$  から  $\vdash v : \tau$ 、E-AWAITDT では既定値  $e_d$  に T-AWAITD の第 2 前提がそのまま使える。

■**E-LET** 逆転補題から  $\vdash v : \tau_1$  と  $x : \tau_1 \vdash e : \tau$ 。  $v$  は値だから補題 13 で任意の文脈で型が付き、補題 15 (代入) から  $\vdash e[x \mapsto v] : \tau$ 。

■**E-SELF** 逆転補題から  $\tau = \text{actor}[c]$ 。 仮定  $\Omega(o) = c$  から T-OREF で  $\vdash \iota_o : \text{actor}[c]$ 。

■**E-REPLYTO** 逆転補題から  $\tau = \text{reply}\langle \rho \rangle$ 。 仮定  $\Phi(k) = \rho$  から T-RREF で  $\vdash \rho_k : \text{reply}\langle \rho \rangle$ 。**タスクの返り値型が、そのタスクの *future* の型と一致しているという仮定 ( $\text{task\_ok}$ ) が、ここで初めて使**

われる。

■E-GET  $\text{heap\_ok}$  の (2) から  $\Sigma(o)$  は値で  $\text{st}(c)$  型。逆転補題から  $\tau = \text{st}(c)$ 。

■E-SET  $H' = (\Omega, \Sigma[o \mapsto v], \Phi, \Psi)$ 。(2) は  $\Omega, \Phi$  が変わらないので自明。(3) は  $\tau = \text{unit}$  で () ゆえ自明。(1) を見る。長さは一点更新で変わらない。 $\Omega(o_2) = c_2$ 、 $\Sigma[o \mapsto v](o_2) = v_2$  とする。 $o_2 = o$  なら  $c_2 = c$ 、 $v_2 = v$  であり、逆転補題から  $\vdash v : \text{st}(c)$ 、 $v$  は値だから補題 13 で任意の文脈でも型が付く。 $o_2 \neq o$  なら更新前と同じで、 $\text{heap\_ok}(H)$  からそのまま従う。

■E-NEW  $H' = (\Omega \cdot c', \Sigma \cdot \text{si}(c'), \Phi, \Psi)$ 。(2) は延長そのもの。(4) は空。(3) は逆転補題から  $\tau = \text{actor}[c']$ 、新しい索引  $|\Omega|$  の値が  $c'$  なので T-OREF。(1) の長さは両方が 1 ずつ伸びるので保たれる。既存の索引の値は変わらないが、型付けは新しい表  $\Omega \cdot c'$  のもとで言い直す必要があり、補題 16 を使う。新しい索引については (P1)(P2) がそのまま「 $\text{si}(c')$  は値で  $\text{st}(c')$  型」を与える。

■E-SEND  $H' = (\Omega, \Sigma, \Phi \cdot \tau_r, \Psi \cdot \perp)$ 、 $\vec{M}' = \{\langle o', m, v, |\Phi| \rangle\}$ 、 $e' = \kappa_{|\Phi|}$ 。逆転補題から  $\vdash \iota_{o'} : \text{actor}[c']$ 、 $\text{mt}(c'', m) = (\tau_a, \tau_r'')$ 、 $\vdash v : \tau_a$ 、 $\tau = \text{future}\langle \tau_r'' \rangle$  が得られ、E-SEND の前提  $\Omega(o') = c'$ 、 $\text{mt}(c', m) = (\tau_a, \tau_r)$  と突き合わせると  $c'' = c'$ 、 $\tau_r'' = \tau_r$  である (表は関数だから)。

- (2):  $\Phi$  の延長。
- (3): 新しい索引  $|\Phi|$  の型が  $\tau_r$  なので T-FREF から  $\vdash \kappa_{|\Phi|} : \text{future}\langle \tau_r \rangle = \tau$ 。
- (1):  $|\Psi \cdot \perp| = |\Phi \cdot \tau_r|$ 。既存の future については  $\Phi$  が伸びただけなので補題 16。新しい索引は  $\Psi$  側が  $\perp$  なので、条件 3 の前提が成り立たず何も要求されない。
- (4): 唯一のメッセージについて、宛先  $o'$  のクラスは  $c'$ 、 $\text{mt}(c', m) = (\tau_a, \tau_r)$ 、引数  $v$  は値で  $\tau_a$  型 (補題 13 と 16)、そして返答先  $|\Phi|$  の型は  $\tau_r$ 。すなわち  $\text{msg\_ok}$ 。

メッセージと future を同時に作るのがこの規則の要点である。「返答先の型は呼んだメソッドの返り値型」という約束が、生成の瞬間に立つ。

■E-AWAIT, E-AWAITDV, E-AWAITRV 逆転補題から  $\vdash \kappa_{k'} : \text{future}\langle \tau \rangle$ 、すなわち  $\Phi(k') = \tau$ 。 $\Psi(k') = v$  (解決済み) だから  $\text{heap\_ok}$  の (3) より  $v$  は値で  $\tau$  型。E-AWAITRV では  $\text{ok}(v) : \text{result}\langle \tau \rangle$  を T-OK で組む。

■E-AWAITRT 逆転補題から  $\tau = \text{result}\langle \tau_0 \rangle$  かつ  $\Phi(k') = \tau_0$ 。規則の前提  $\Phi(k') = \tau'$  と合わせて  $\tau' = \tau_0$  だから、 $e' = \text{err}_{\tau_0}$  は T-ERR で  $\text{result}\langle \tau_0 \rangle = \tau$  型を持つ。err が型注釈を持っているのは、ここで型を失わないためである。

■E-ANSWER  $H' = (\Omega, \Sigma, \Phi, \Psi[k' \mapsto v])$ 。(2)(3)(4) は自明 ( $\tau = \text{unit}$ )。(1) を見る。長さは一点更新で変わらない。 $\Psi$  の索引  $k_2$  について、 $k_2 \neq k'$  なら更新前と同じ。 $k_2 = k'$  のときは、逆転補題から  $\vdash \rho_{k'} : \text{reply}\langle \tau_0 \rangle$  すなわち  $\Phi(k') = \tau_0$ 、かつ  $\vdash v : \tau_0$  であり、 $v$  は値だから補題 13 で任意の文脈でも  $\tau_0$  型。 $\Phi(k_2) = \Phi(k') = \tau_0$  なので条件 3 を満たす。

これが委譲の安全性の核心である。書き込む主体は future  $k'$  の持ち主でなくてよい。書き込む権利は値  $\rho_{k'}$  が表しており、その型  $\text{reply}\langle \tau_0 \rangle$  は  $\Phi$  から来ている。 $\Phi$  は伸びるだけで書き換わらないから、 $\rho_{k'}$  がどこへ運ばれても  $\tau_0$  は変わらない。したがって「誰が答えても、入る値の型は同じ」。

■E-CTX 帰納法の仮定から  $\text{heap\_ok}(H')$ 、表の延長、 $\vdash e'_0 : \tau'$ 、メッセージの  $\text{msg\_ok}$  が得られる。補題 17 (文脈の置換) を、延長した表  $\Omega', \Phi'$  のもとで適用すれば  $\vdash E[e'_0] : \tau$  を得る。□

注意 20 (機械証明との違い). Coq の開発では評価文脈を導入せず、合同規則を 24 本並べてある。文脈の文法 (§4.2) の各生成規則が合同規則 1 本に対応し、補題 17 は各合同規則の証明の中に溶けている。どちらも証明の中身は同じだが、機械証明では文脈の「穴」を扱う補題を書かずに済むぶん、規則を並べる方が短い。

## 9 主定理

**定理 21** (メソッド不理解は起きない).  $\text{conf\_ok}(H, \vec{M}, \vec{t})$  で  $\langle o, m, v, k \rangle \in \vec{M}$  ならば、ある  $c, \tau_a, \tau_r$  があって  $\Omega(o) = c$ ,  $\text{mt}(c, m) = (\tau_a, \tau_r)$ ,  $v$  は  $\tau_a$  型、 $\Phi(k) = \tau_r$ 。

*Proof.*  $\text{msg\_ok}$  の定義そのものである。飛んでいるメッセージが常にこの形であることは、定理 22 (保存) が保証する。□

**定理 22** (保存).  $\text{conf\_ok}(C)$  かつ  $C \Longrightarrow C'$  ならば  $\text{conf\_ok}(C')$ 。

*Proof.*  $C \Longrightarrow C'$  の導出で場合分けする。

■C-TASK 動いたタスク  $\langle o, k, e \rangle$  について  $\text{task\_ok}$  から  $\Omega(o) = c$ ,  $\Phi(k) = \tau$ ,  $\Omega; \Phi; c; \tau; \emptyset \vdash e : \tau$  を得る。 $\rho := \tau$  として補題 19 (局所保存) を適用すると、 $\text{heap\_ok}(H')$ ,  $\Omega \sqsubseteq \Omega'$ ,  $\Phi \sqsubseteq \Phi'$ ,  $\Omega'; \Phi'; c; \tau; \emptyset \vdash e' : \tau$ , そして新しいメッセージが  $\text{msg\_ok}$  であることが得られる。

- $\Omega_0 \sqsubseteq \Omega \sqsubseteq \Omega'$  は推移律。
- 元からあったメッセージとタスクは、表が伸びただけなので補題 16 と索引の保存から、そのまま  $H'$  のもとでも  $\text{msg\_ok}/\text{task\_ok}$ 。
- 動いたタスクは  $\langle o, k, e' \rangle$  で、 $\Omega'(o) = c$ ,  $\Phi'(k) = \tau$  (索引は保存される)、型付けは上で得た。

■C-FINISH 終わったタスク  $\langle o, k, v \rangle$  について  $\text{task\_ok}$  から  $\Phi(k) = \tau$ ,  $\vdash v : \tau$ 。  $v$  は値だから補題 13。  $\Psi$  の  $k$  番だけが  $v$  になるので、 $\text{heap\_ok}$  の条件 3 は  $k$  番について  $\vdash v : \Phi(k) = \tau$  から従い、他の番はそのまま。  $\Omega, \Phi$  は変わらないので、残りのメッセージとタスクは影響を受けない。(  $\text{msg\_ok}$  も  $\text{task\_ok}$  も  $\Psi$  を見ないことに注意。)

■C-DELIVER 配送するメッセージ  $\langle o, m, v, k \rangle$  について  $\text{msg\_ok}$  から  $\Omega(o) = c$ ,  $\text{mt}(c, m) = (\tau_a, \tau_r)$ ,  $v$  は値で  $\tau_a$  型、 $\Phi(k) = \tau_r$ 。 ヒープは変わらないので  $\text{heap\_ok}$  はそのまま。 新しいタスク  $\langle o, k, \text{mb}(c, m)[0 \mapsto v] \rangle$  が  $\text{task\_ok}$  であることを言えばよい。 整合性条件 (P3) を  $\Omega := \Omega$ ,  $\Phi := \Phi$  に適用して

$$\Omega; \Phi; c; \tau_r; \{0:\tau_a\} \vdash \text{mb}(c, m) : \tau_r$$

を得る ((P3) の前提  $\Omega_0 \sqsubseteq \Omega$  は  $\text{conf\_ok}$  の一部である)。  $v$  は値で任意の文脈で  $\tau_a$  型だから、補題 15 (代入) より  $\Omega; \Phi; c; \tau_r; \emptyset \vdash \text{mb}(c, m)[0 \mapsto v] : \tau_r$ 。  $\Phi(k) = \tau_r$  だから、判断の第 4 成分と結論の型がともに  $\Phi(k)$  になっており、 $\text{task\_ok}$  の形をしている。 □

**系 23.**  $\text{conf\_ok}(C)$  かつ  $C \Longrightarrow^* C'$  ならば  $\text{conf\_ok}(C')$ 。

*Proof.*  $\Longrightarrow^*$  の長さに関する帰納法と定理 22。 □

**定理 24** (進行).  $\text{conf\_ok}(C)$  ならば、 $\text{terminal}(C)$  か、 $\exists C'. C \Longrightarrow C'$  か、 $\text{blocked}(C)$  のいずれかが



成り立つ。

*Proof.*  $C = (H, \vec{M}, \vec{t})$  とする。

$\vec{M} \neq \emptyset$  のとき。先頭のメッセージ  $\langle o, m, v, k \rangle$  は `msg_ok` だから  $\Omega(o) = c$  と  $\text{mt}(c, m) = (\tau_a, \tau_r)$  が存在する。これは C-DELIVER の前提そのものなので、必ず配送できる。「宛先にそのメソッドが無くて配送できない」が起こらないのが、ここである。

$\vec{M} = \emptyset$  のとき。タスク列を先頭から見る。各タスクは `task_ok` だから、補題 18 (局所進行) が使える。値になっているタスクがあれば C-FINISH で、一步進めるタスクがあれば C-TASK で、構成が一步進む。どのタスクも値でも一步進めるものでもないなら、局所進行の (iii) からすべてのタスクが `awaiting` である。タスク列が空なら `terminal`、空でなければ `blocked` である。□

**定理 25** (型安全性).  $\text{conf\_ok}(C)$  かつ  $C \Longrightarrow^* C'$  ならば  $\neg \text{stuck}(C')$ 。

*Proof.* 系 23 から  $\text{conf\_ok}(C')$ 。定理 24 から  $C'$  は終状態か、一步進めるか、待ち状態である。`stuck` の定義 (定義 5) はこの三つをすべて否定するので、矛盾する。□

注意 26 (この型安全性が排除しているもの). 定理 25 が排除しているのは、たとえば次のような状態である。

- 宛先にメソッドが無いメッセージが、配送できずに残る (メソッド不理解)。
- `true + 1` のように、型の合わない基底規則の当て先が無い式が残る。
- 存在しない actor 番号・future 番号を指す参照が残る。
- `err` から値を取り出そうとして落ちる (既定値を必須にしたので起きない)。
- 解決済みの future を二度書いて型が変わる (E-ANSWER は型を変えない)。

排除していないのは、`blocked` — 全員が期限なしの待ちに入って止まることだけである。

## 10 派生する不変条件

**定理 27** (状態の型は保たれる).  $\text{conf\_ok}(C)$ 、 $C \Longrightarrow^* (H', \vec{M}', \vec{t}')$ 、 $\Omega'(o) = c$ 、 $\Sigma'(o) = v$  ならば、 $v$  は値であり、任意の文脈で  $\vdash v : \text{st}(c)$ 。

*Proof.* 系 23 と `heap_ok` の条件 2。□

**定理 28** (future に入る値の型は保たれる).  $\text{conf\_ok}(C)$ 、 $C \Longrightarrow^* (H', \vec{M}', \vec{t}')$ 、 $\Phi'(k) = \tau$ 、 $\Psi'(k) = v$  ならば、 $v$  は値であり、任意の文脈で  $\vdash v : \tau$ 。

*Proof.* 系 23 と `heap_ok` の条件 3。□

定理 28 は、この断片でいちばん強い主張である。future  $k$  に値が入る経路は三つある — 自分の本体が値になって C-FINISH で入るか、自分が `answer replyto v` で入れるか、返答先を渡された他人が `answer  $\rho_k v$`  で入れるか。三つ目は委譲であり、AIPL<sup>-2</sup> では書けない。それでも、**入る値の型は  $\Phi(k)$  のままである**。待つ側は `now  $\kappa_k : \Phi(k)$`  と型付けされているので、誰が答えたかによらず、受け取る型は変わらない。

**定理 29** (期限付きの待ちは必ず一步進める).  $\Phi(k') = \tau$  ならば  $H; o; k \vdash \text{now } \kappa_{k'} \text{ timeout } n \text{ else } e_d \longrightarrow$

$H; \emptyset; e_d$ 。とくに、この式は待ち状態ではない —  $\neg \text{awaiting}(H, \text{now } \kappa_{k'} \text{ timeout } n \text{ else } e_d)$ 。

*Proof.* 前者は E-AWAITDT そのもの。後者は `awaiting` の定義（定義 5）による —  $E[\text{now } \kappa]$  の形に書けるのは、期限付きの待ちの**主部**が待っている場合だけであり、主部が  $\kappa_{k'}$ （値）ならその形にならない。□

**系 30** (すべての待ちに期限をつけたなら). 構成  $C$  のすべてのタスクの式が、期限のない `now` を含まないならば、 $\neg \text{blocked}(C)$ 。したがって定理 24 は二択になる。

*Proof.* `awaiting` の定義は期限のない `now` を必ず一つ名指しする。□

これが第 2 版で紙の上だけ示していた主張の、意味論の側での対応物である。ただし得られるのはデッドロックが有限時間の失敗に変わることであって、詰まらないことではない — 互いに待ち合う二者は、両方が期限切れになる。

## 第 III 部

# 機械証明

## 11 開発の構成

証明系は **Rocq (Coq) 9.1.0** (OCaml 5.1.1 でビルド) である。本稿に対応するファイルは二つ、いずれも `yaskodama/abclcp` の `coq/` にある。

| ファイル                                   | 内容                       | 行数   |
|----------------------------------------|--------------------------|------|
| <code>AIPLSoundnessMax.v</code>        | 構文・型・意味論・不変条件・全補題・全定理    | 1510 |
| <code>AIPLSoundnessMaxExample.v</code> | 例題プログラム・整合性の確認・デッドロックの反例 | 214  |

同じディレクトリにある `AIPLSoundness.v` (第 1 版)、`AIPLSoundness2.v` (第 3 版、義務レベルと効果)、`AIPLDining.v` (食事する哲学者の資源順序) とは独立で、互いに `Require` していない。

## 12 紙と機械の対応

| 本稿                                                 | <code>AIPLSoundnessMax.v</code>              | 行   |
|----------------------------------------------------|----------------------------------------------|-----|
| 型 $\tau$                                           | <code>Inductive ty</code>                    | 48  |
| 式 $e$                                              | <code>Inductive tm</code>                    | 59  |
| 値 $v$                                              | <code>Inductive value</code>                 | 101 |
| 代入 $e[x \mapsto v]$                                | <code>Fixpoint subst</code>                  | 113 |
| ヒープ $H = (\Omega, \Sigma, \Phi, \Psi)$             | <code>Record heap {hot;hst;hft;hfv}</code>   | 143 |
| メッセージ・タスク・構成                                       | <code>msg / task / conf</code>               | 150 |
| 表の延長 $\sqsubset$                                   | <code>ext</code>                             | 180 |
| <code>st, si, mt, mb, <math>\Omega_0</math></code> | <code>Variable stype sinit mtab mbody</code> | 248 |
|                                                    | <code>ot0</code>                             |     |

| 本稿                                                  | AIPLSoundnessMax.v                   | 行       |
|-----------------------------------------------------|--------------------------------------|---------|
| $\Omega; \Phi; c; \rho; \Gamma \vdash e : \tau$     | ht ot ft C R G e T                   | 268     |
| 整合性 (P1)(P2)(P3)                                    | sinit_value / sinit_ok / bodies_ok   | 323     |
| 局所簡約 $H; o; k \vdash e \rightarrow H'; \vec{M}; e'$ | tstep H o k e H' out e'              | 444     |
| 評価文脈 $E$                                            | 合同規則 ST*1, ST*2 (24 本)               | 511     |
| awaiting                                            | Inductive awaiting                   | 572     |
| $\Rightarrow$ 、 $\Rightarrow^*$                     | cstep / csteps                       | 600     |
| heap_ok                                             | heap_ok                              | 623     |
| msg_ok                                              | msg_ok                               | 632     |
| task_ok                                             | task_ok                              | 643     |
| conf_ok                                             | conf_ok                              | 652     |
| terminal, blocked, stuck                            | terminal / blocked / stuck           | 659     |
| 補題 11 (逆転)                                          | ht_oref_inv ... ht_answer_inv (27 本) | 861–990 |
| 補題 12 (標準形)                                         | canon_int ... canon_res (8 本)        | 397–441 |
| 補題 13 (値は文脈に依らない)                                   | value_ht_indep                       | 360     |
| 補題 15 (代入)                                          | substitution                         | 369     |
| 補題 16 (単調性)                                         | ht_mono (ht_env_ext も)               | 349     |
| 補題 17 (文脈の置換)                                       | 合同規則の各場合に溶けている                       | —       |
| 補題 18 (局所進行)                                        | local_progress                       | 673     |
| 補題 19 (局所保存)                                        | local_preservation                   | 994     |
| (表の伸長に対する不変条件の持ち上げ)                                 | msg_ok_mono / task_ok_mono           | 1301    |
| 定理 21 (メソッド不理解なし)                                   | no_method_not_understood             | 1330    |
| 定理 22 (保存)                                          | preservation                         | 1345    |
| 系 23                                                | preservation_star                    | 1408    |
| 定理 24 (進行)                                          | progress (tasks_progress も)          | 1440    |
| 定理 25 (型安全性)                                        | type_safety                          | 1466    |
| 定理 27 (状態の型)                                        | state_type_invariant                 | 1474    |
| 定理 28 (future の型)                                   | future_type_invariant                | 1487    |
| 定理 29 (期限)                                          | timeout_always_progresses            | 1501    |
|                                                     | timeout_never_awaits                 | 1506    |

注意 31 (機械証明の側でだけ必要になったもの). 紙では暗黙に済ませたが、機械証明では明示した補題が三種類ある。

- 表の索引に関する補題 (nth\_upd\_eq, nth\_app\_last, nth\_app1\_inv など)。「一点更新は他の索引を変えない」「後ろへの延長は既存の索引を変えない」を、リストの操作として書いたものである。
- 型環境の外延性 (ht\_env\_ext)。紙では  $\Gamma$  を集合のように扱ったが、機械証明では関数なので、「値が等しい二つの環境は同じ型付けを与える」を補題にする必要がある。

- 不変条件の持ち上げ (`msg_ok_mono`, `task_ok_mono`)。紙では「補題 16 からそのまま」と書いた箇所である。

## 13 仮定が空虚でないこと

すべての定理は整合性 (P1)(P2)(P3) を仮定している。そこで、 $\text{AIPL}^{-2}$  では書けない機構（返答の委譲）を実際に使うプログラムを一つ与え、三条件を満たすことを機械検査した。

```
class Worker {                                // クラス 0
  state : int = 0
  method add(x : int) : int { var y = state + x; state := y; y }
  method serve(r : reply<int>) : unit { answer r 7 } // 他人の返答先に答える
}
class Front {                                 // クラス 1
  method ask(x : int) : int {
    var f = future w.serve(replyto);          // 自分の返答先を渡してしまう
    0
  }
}
```

`Front.ask` は自分の返答先 `replyto` を `Worker` へ渡す。`Worker.serve` は、自分が呼ばれた future ではなく、**渡された返答先**へ答える。 $\text{AIPL}^{-2}$  の義務レベルでは、返す義務がレベルの高い側から低い側へ移るので、これは書けない。 $\text{AIPL}_{\max}^{-}$  では T-REPLYTO と T-ANSWER で型が付く — `Front.ask` の返り値型が `int` だから  $\rho = \text{int}$ 、よって `replyto : reply<int>` であり、`Worker.serve` の引数型 `reply<int>` と一致する。

Coq 側の対応は次のとおりである (`AIPLSoundnessMaxExample.v`)。

| 名前                                                                                                                 | 内容                                                    | 行     |
|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|-------|
| <code>ex_stype</code> , <code>ex_sinit</code> , <code>ex_mtab</code> , <code>ex_mbody</code> , <code>ex_ot0</code> | 上のプログラム                                               | 29–51 |
| <code>ex_sinit_value</code>                                                                                        | (P1)                                                  | 53    |
| <code>ex_sinit_ok</code>                                                                                           | (P2)                                                  | 56    |
| <code>ex_bodies_ok</code>                                                                                          | (P3)。三つの本体すべてに型が付く                                    | 60    |
| <code>ex_conf_ok</code>                                                                                            | <code>Front.ask</code> が走っている構成が <code>conf_ok</code> | 97    |
| <code>ex_can_step</code>                                                                                           | その構成は定理 24 により一歩進める                                   | 124   |
| <code>ex_never_stuck</code>                                                                                        | そこから到達できるとの構成も行き詰まらない                                 | 141   |

## 14 公理なしの確認と再現手順

```
cd ~/aios/abclcp/coq
make                # ABCM, AIPLSoundness, AIPLSoundness2, AIPLSoundness2Example,
                   # AIPLDining, AIPLSoundnessMax, AIPLSoundnessMaxExample, ABCMEmbedding
```

make check は、既存の 25 本に本稿の 15 本を加えた 40 本すべてについて

Closed under the global context

を出力する。すなわち公理を一つも使っていない。本稿の 15 本は次のものである。

|                                            |                      |
|--------------------------------------------|----------------------|
| AIPLSoundnessMax.no_method_not_understood  | 定理 21                |
| AIPLSoundnessMax.preservation              | 定理 22                |
| AIPLSoundnessMax.preservation_star         | 系 23                 |
| AIPLSoundnessMax.progress                  | 定理 24                |
| AIPLSoundnessMax.type_safety               | 定理 25                |
| AIPLSoundnessMax.state_type_invariant      | 定理 27                |
| AIPLSoundnessMax.future_type_invariant     | 定理 28                |
| AIPLSoundnessMax.timeout_always_progresses | 定理 29                |
| ex_bodies_ok, ex_conf_ok                   | 整合性 (P3) と初期構成       |
| ex_can_step, ex_never_stuck                | 例題が動くこと・行き詰まらないこと    |
| max_admits_deadlock                        | §18: 型が付いてデッドロックする構成 |
| max_is_not_deadlock_free                   | §18: デッドロック自由の否定     |
| dl_not_stuck                               | §18: それでも行き詰まってはいいない |

Section の Variable/Hypothesis は End で各定理の前提に変わる。したがって st, si, mt, mb,  $\Omega_0$  と (P1)(P2)(P3) は公理ではなく仮定であり、§13 の例題がその仮定を実際に満たしている。

## 第 IV 部

# なぜこれが上限か

「最大」は、次の三つを合わせて主張する。

1. 加えた機構がすべて入っていること (§15)。
2. さらに加えると健全性・安全性が**実際に**破れる機構があり、その破れ方が分かっていること (§16)。
3. この断片ではデッドロック自由が**実際に**破れること——つまり、これ以上広げないことと引き換えに得られる性質は無く、逆に狭めれば得られること (§18)。

## 15 加えた機構

| 機構         | AIPL の記法 | 健全性を壊さない理由                                                                      |
|------------|----------|---------------------------------------------------------------------------------|
| 無条件の await | now f    | 進行定理の第 3 の枝 (待ち) に落ちるだけで、行き詰まりにはならない。待つ相手の型は $\Phi$ が決めており、値が入るなら必ずその型 (定理 28)。 |
| 文字列と ++    | "a" ++ s | 基底型を一つ増やし、演算を型で分けただけ。標準形補題に一行増える。                                               |

| 機構                                | AIPL の記法                              | 健全性を壊さない理由                                                                                                                                |
|-----------------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 対                                 | $m(a, b)$ 、複数フィールド                    | 値の構成子が増えるが、値の型付けは文脈に依らないまま（補題 13 の帰納段）。                                                                                                   |
| $\text{result}\langle\tau\rangle$ | <code>ok / err / is_ok / value</code> | <code>value</code> に既定値を必須にしたので、失敗した <code>result</code> からの取り出しが全域関数になる。範囲外・失敗が行き詰まりにならない形になっている。                                        |
| 期限付きの待ち                           | <code>now f timeout 500 else d</code> | 既定値の型を本体と同じにしたので、どちらの枝でも型が変わらない。しかも常に一歩進めるので、進行が強くなる（定理 29）。                                                                              |
| 期限付きの待ち（result 版）                 | <code>now f timeout 500</code>        | 失敗側は $\text{err}_\tau$ 。 $\tau$ は $\Phi(k)$ から取るので型が定まる。                                                                                  |
| 一級の返答先                            | <code>replyto / answer r v</code>     | <code>replyto</code> は値ではなく、 <b>自分のところで</b> $\rho_k$ に簡約される。 $\rho_k$ の型は $\Phi$ が決めるので、どこへ運ばれても変わらない（補題 13 の後の注意、および局所保存の E-ANSWER の場合）。 |
| 自己送信・待ちの環                         | <code>now self.m(x)</code>            | 型システムは何も禁じない。止まるかどうかは型の問題ではない。                                                                                                            |
| 孤児 future                         | —                                     | 誰も埋めない future があっても、 <code>heap_ok</code> の条件 3 は「入っているなら型が合う」としか言わない。                                                                    |

## 16 加えると壊れるもの

以下は、AIPL の実装にはあるが  $\text{AIPL}_{\max}^-$  に入れなかった機構である。入れなかった理由は**健全性が安全性が実際に破れるから**であり、破れ方をそれぞれ示す。合わせて、**どう直せば入るかも書く** — そのほとんどは、実装がすでにその形をとっている。

### 16.1 動的な宛先の remote

`remote(host, name)` の `host`、`name` が実行時に計算される文字列だと、宛先 actor のクラスが型から決まらない。すると T-SEND の第 2 前提  $\text{mt}(c', m) = (\tau_a, \tau_r)$  が立たず、送信式に型を与えられない。無理に「宛先は `actor[?]`」として型を付けると、C-DELIVER の前提が満たせない構成が到達可能になり、配送できないメッセージが残る — すなわち `stuck`。**メソッド不理解が復活する。**

直し方は二つある。(a) 宛先を静的な文字列リテラルに限る（現在の実装がこれである。`remote(host, actor)` は文字列リテラルしか受けない）。(b) 遠隔呼び出しの型を `result⟨τ⟩` にして、宛先が無い・メソッドが無いを実行時の失敗として値で返す。 $\text{AIPL}_{\max}^-$  には `result⟨τ⟩` があるので、この形なら**そのまま入る**。

## 16.2 any と動的キャスト

any を入れて  $\text{any} \rightarrow \text{int}$  の暗黙変換を許すと、標準形補題 12 が壊れる。 $\vdash v : \text{int}$  なのに  $v$  が数でない、という状況が起こるので、局所進行の T-ADD の場合で基底規則が当たらない。 $\text{true} + 1$  が残る。直し方は、キャストを  $\text{result}(\tau)$  を返す検査つきの操作にすること。

## 16.3 計算されたメソッド名

$\text{future } t.(f(x))(v)$  のようにメソッド名を計算すると、§16.1 と同じ理由で T-SEND が書けない。実装 AIPL はメソッド名を識別子に限定している。

## 16.4 範囲検査のない添字

配列  $a[i]$  を全域でない操作として入れると、 $i$  が範囲外のとき当てる規則が無く stuck になる。AIPL<sub>max</sub> が value に既定値を必須にしたのと同じ手当てが要る — 既定値付きの  $a[i] \text{ else } d$  か、 $\text{result}(\tau)$  を返す添字にすれば入る。(実装の配列は複製で更新される値であり、array\_get は範囲外で失敗する。ここは実装の側に手当てが要る箇所である。)

## 16.5 注釈のない reply

メソッドの返り値型が宣言されていないと、(P3) の  $\rho$  と  $\tau_r$  が決まらない。すると replyto に型を付けられず、task\_ok の「判断の第 4 成分と結論の型がともに  $\Phi(k)$ 」という形が作れない。第 1 版がまさにこの点で詰まり、web\_listen / web\_expose から到達できるメソッドには返り値型注釈を必須にするという実装上の規則になった。本稿の (P3) は、その規則の形式化である。

## 16.6 replyto を値にすること

これは細かいが本質的である。replyto 自身を値にして他の actor へ送れるようにすると、補題 13 (値の型付けは文脈に依らない) が壊れる — replyto の型は  $\rho$  に依存するからである。補題 13 は代入補題と heap\_ok の両方が依存しているので、ここが壊れると保存定理まで壊れる。AIPL<sub>max</sub> は replyto を式に留め、簡約の結果である  $\rho_k$  (型は  $\Phi$  由来) だけを運べる値にした。一級の返答先が安全に入るのは、この一段のおかげである。

## 16.7 受け取る型が動的に決まる選択受信

select / case で受け取るメッセージの型が実行時に決まると、msg\_ok の「引数は宣言された引数型を持つ」が言えなくなり、C-DELIVER で起こすタスクに型が付かない。受信の形をメソッドごとに固定すれば (実装がそうしている) 問題は起きない。

## 17 直交する層

次のものは、健全性を壊さないが本稿では扱っていない。いずれも AIPL<sub>max</sub> の上に層として載る。

**効果注釈**  $\{!act, mut, ai, \dots\}$  第3版 (AIPLSoundness2.v) で機械検証済みである。future( $\tau$ ) の型に効果を載せ、送信は引き継がず、待ちが引き継ぐという規則を与えると、効果健全性が保存定理から出る。AIPL<sub>max</sub><sup>-</sup> に載せるには、future( $\tau$ ) を future( $\tau!E$ ) に置き換えるだけでよく、本稿のどの証明も構造は変わらない。

**義務レベル** @n 第3版の主題であり、狭める方向の層である。AIPL<sub>max</sub><sup>-</sup> に載せると、載せた分だけ書けるプログラムが減り、代わりにデッドロック自由が得られる。

**資源順序** (acquire/release) AIPLDining.v で別に扱っている。型ではなく、実行時の状態遷移についての議論である。

**分散配備** (node\_allow / deploy / source\_of) 型に関係しない。どのノードで走るかは、本稿の意味論では見えない。

**浮動小数・AI 呼び出し** 前者は基底型を増やすだけ。後者は「str を返す不透明な関数」として入る — 中身が何であれ、型としては文字列である。

## 18 デッドロック自由は、ここでは成り立たない

最後に、AIPL<sub>max</sub><sup>-</sup> が上限であることのいちばん直接的な証拠を挙げる。

**命題 32** (型が付いてデッドロックする構成がある). §13 のプログラムのもとで、次の構成  $C_{dl}$  は  $conf\_ok(C_{dl})$  かつ  $blocked(C_{dl})$  である。

$$\begin{aligned} H_{dl} &= ([0; 1], [0; 0], [\text{int}; \text{int}], [\perp; \perp]) \\ C_{dl} &= (H_{dl}, \emptyset, [\langle 0, 0, \text{now } \kappa_1 \rangle, \langle 1, 1, \text{now } \kappa_0 \rangle]) \end{aligned}$$

*Proof.* heap\_ok: 長さは合っており、状態はどちらも  $0 : \text{int} = \text{st}(c)$ 、解決済みの future は無い。 $\Omega_0 \sqsubseteq \Omega$  は等号。メッセージは無い。task\_ok: 第1のタスクは  $\Omega(0) = 0$ 、 $\Phi(0) = \text{int}$  で、T-FREF と T-AWAIT から  $\Omega; \Phi; 0; \text{int}; \emptyset \vdash \text{now } \kappa_1 : \text{int}$ 。第2のタスクも同様。よって conf\_ok。blocked: メッセージ列は空、タスク列は空でなく、 $\Psi(0) = \Psi(1) = \perp$  だから両方のタスクが awaiting (文脈は []).  $\square$

actor 0 のタスクは future 1 を待ち、actor 1 のタスクは future 0 を待つ。それぞれの future には埋める者がいる (相手のタスクである) — AIPL<sup>-2</sup> の生産者不変条件は満たされている。それでも止まる。止めているのは待ちの向きであり、それを禁じるのが AIPL<sup>-2</sup> の義務レベルであった。AIPL<sub>max</sub><sup>-</sup> にはそれが無い。

**系 33.** AIPL<sub>max</sub><sup>-</sup> では、定理 24 (進行) の第 3 の枝を落とせない。すなわち  $\forall C. conf\_ok(C) \Rightarrow \neg blocked(C)$  は偽である。

**系 34** (それでも壊れてはいない).  $\neg stuck(C_{dl})$ 。

*Proof.* 定理 25 を  $C_{dl} \Longrightarrow^* C_{dl}$  (0 歩) に適用する。あるいは直接、blocked( $C_{dl}$ ) と stuck の定義から。  $\square$

この最後の系が、§1.2 で述べた線引きの意味である。 $C_{dl}$  は壊れていない — 型はすべて合っており、未定義の操作も、理解されないメッセージも、宙に浮いた参照も無い。ただ答えが来ないだけである。型システムに答えられるのはここまでで、その先 (答えが来るかどうか) は義務レベルのような別の道具の仕事になる。

三つの断片の到達点を、改めて並べておく。



|              | AIPL <sup>-</sup> | AIPL <sup>-2</sup> | AIPL <sup>-</sup> <sub>max</sub> |
|--------------|-------------------|--------------------|----------------------------------|
| 保存           | 定理                | 定理                 | 定理 22                            |
| 進行           | 定理 (await なし)     | 定理 (二択)            | 定理 24 (三択)                       |
| 型安全性         | 定理                | 定理                 | 定理 25                            |
| デッドロック自由     | await なしの断片のみ     | 定理                 | 反例 (命題 32)                       |
| 効果健全性        | —                 | 定理                 | 層として載る                           |
| 委譲・期限・result | 書けない              | 書けない               | 書ける                              |

## 19 まとめ

1. 目標を型健全性と型安全性の二つに絞ると、AIPL<sup>-2</sup> から義務レベルと生産者不変条件を外し、文字列・対・result( $\tau$ )・期限付きの待ち・一級の返答先を加えた断片 AIPL<sup>-</sup><sub>max</sub> まで広げられる。
2. その範囲で、保存・進行・型安全性・状態の型不変・future の型不変・期限の進行を、紙の上で証明し、Rocq 9.1.0 で機械検証した。新規 15 定理、いずれも公理なし。
3. 一級の返答先を安全にしているのは、replyto を値にせず、 $\rho_k$  を値にするという一段である。返答先の型は  $\Phi$  が決めるので、どこへ運ばれても変わらない。誰が答えても、future に入る値の型は同じ (定理 28)。
4. これ以上広げると壊れるものは特定できている (§16)。そのほとんどについて、実装 AIPL はすでに壊れない形をとっている — 静的な remote の宛先、識別子に限ったメソッド名、戻り値型注釈の強制。本稿の (P3) はその規則の形式化である。
5. そして AIPL<sup>-</sup><sub>max</sub> では、デッドロック自由は成り立たない (命題 32)。型が付いたまま止まる構成が実在する。デッドロック自由が要るなら、AIPL<sup>-2</sup> の義務レベルまで戻って言語を狭めるほかない。これが選択の全体像である。